

SQL Part 5: INSERT, UPDATE, DELETE & CREATE TABLE Made Easy — Free Practice Guide

Beyond School · Ready to learn in a fun way! Anyone can learn — easy! Nothing to install, just a web browser.

Step 1: Open a free online SQL playground

- sqliteonline.com — choose "SQLite", paste, click Run.
- db-fiddle.com — pick "SQLite" as the database, paste and Run.
- sqlime.org — a simple SQLite editor in your browser.
- sqlbolt.com — free interactive lessons for extra practice.

Step 2: Paste this setup script and click Run

It creates the same library tables used in the video. If anything goes wrong, just run it again in a fresh playground.

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  title TEXT,
  author TEXT,
  year INTEGER,
  genre TEXT,
  pages INTEGER
);
INSERT INTO books VALUES
  (1, 'Harry Potter', 'J.K. Rowling', 1997, 'Fantasy', 309),
  (2, 'Matilda', 'Roald Dahl', 1988, 'Comedy', 240),
  (3, 'Wonder', 'R.J. Palacio', 2012, 'Drama', 310),
  (4, 'The Hobbit', 'J.R.R. Tolkien', 1937, 'Fantasy', 310),
  (5, 'Holes', 'Louis Sachar', 1998, 'Mystery', 233),
  (6, 'Charlotte''s Web', 'E.B. White', 1952, 'Classic', 192),
  (7, 'Dog Man', 'Dav Pilkey', 2016, 'Comedy', 240),
  (8, 'Coraline', 'Neil Gaiman', 2002, 'Fantasy', 162);

CREATE TABLE members (
  member_id INTEGER PRIMARY KEY,
  name TEXT,
  city TEXT,
  joined TEXT,
```

```
  email TEXT
);
INSERT INTO members VALUES
  (1, 'Aisha', 'Austin', '2023-01-15', 'aisha@mail.com'),
  (2, 'Ben', 'Denver', '2023-03-02', NULL),
  (3, 'Carlos', 'Austin', '2024-06-20', 'carlos@mail.com'),
  (4, 'Dana', 'Boston', '2024-09-10', NULL),
  (5, 'Eli', 'Denver', '2025-02-01', 'eli@mail.com');

CREATE TABLE loans (
  loan_id INTEGER PRIMARY KEY,
  book_id INTEGER,
  member_id INTEGER,
  loan_date TEXT,
  return_date TEXT
);
INSERT INTO loans VALUES
  (1, 1, 1, '2026-01-05', '2026-01-19'),
  (2, 3, 1, '2026-02-10', NULL),
  (3, 2, 2, '2026-01-20', '2026-02-03'),
  (4, 4, 3, '2026-03-01', '2026-03-15'),
  (5, 1, 3, '2026-04-02', NULL),
  (6, 7, 4, '2026-04-10', '2026-04-24'),
  (7, 7, 2, '2026-05-01', NULL),
  (8, 5, 4, '2026-05-15', NULL);
```

Step 3: Run every query from the video, in order

Paste one at a time, click Run, and compare with the expected result.

INSERT: add a book

```
INSERT INTO books VALUES (9, 'The BFG', 'Roald Dahl',         1982, 'Fantasy', 208);
SELECT * FROM books WHERE id >= 6;
```

```
Expected result:
id | title | author | year | genre | pages
6 | Charlotte's Web | E.B. White | 1952 | Classic | 192
7 | Dog Man | Dav Pilkey | 2016 | Comedy | 240
8 | Coraline | Neil Gaiman | 2002 | Fantasy | 162
9 | The BFG | Roald Dahl | 1982 | Fantasy | 208
```

INSERT with column names

```
INSERT INTO members (member_id, name, city) VALUES (6, 'Farah', 'Austin');
SELECT * FROM members;
```

```
Expected result:
member_id | name | city | joined | email
1 | Aisha | Austin | 2023-01-15 | aisha@mail.com
2 | Ben | Denver | 2023-03-02 | NULL
3 | Carlos | Austin | 2024-06-20 | carlos@mail.com
4 | Dana | Boston | 2024-09-10 | NULL
5 | Eli | Denver | 2025-02-01 | eli@mail.com
6 | Farah | Austin | NULL | NULL
```

UPDATE with WHERE

```
UPDATE members SET email = 'farah@mail.com' WHERE member_id = 6;
SELECT * FROM members WHERE member_id = 6;
```

```
Expected result:
member_id | name | city | joined | email
6 | Farah | Austin | NULL | farah@mail.com
```

UPDATE a loan

```
UPDATE loans SET return_date = '2026-06-01' WHERE loan_id = 2;
SELECT * FROM loans WHERE loan_id = 2;
```

Expected result:
loan_id | book_id | member_id | loan_date | return_date
2 | 3 | 1 | 2026-02-10 | 2026-06-01

DELETE with WHERE

```
DELETE FROM loans WHERE loan_id = 8;
SELECT * FROM loans;
```

Expected result:
loan_id | book_id | member_id | loan_date | return_date
1 | 1 | 1 | 2026-01-05 | 2026-01-19
2 | 3 | 1 | 2026-02-10 | 2026-06-01
3 | 2 | 2 | 2026-01-20 | 2026-02-03
4 | 4 | 3 | 2026-03-01 | 2026-03-15
5 | 1 | 3 | 2026-04-02 | NULL
6 | 7 | 4 | 2026-04-10 | 2026-04-24
7 | 7 | 2 | 2026-05-01 | NULL

CREATE TABLE

```
CREATE TABLE reviews (    review_id INTEGER PRIMARY KEY,    book_id    INTEGER NOT NULL REFERENCES books(id),    reviewer  TEXT NOT NULL,    stars     INTEGER,    UNIQUE (book_id, reviewer) );
SELECT name, type, "notnull", pk FROM pragma_table_info('reviews');
```

Expected result:
name | type | notnull | pk
review_id | INTEGER | 0 | 1
book_id | INTEGER | 1 | 0
reviewer | TEXT | 1 | 0
stars | INTEGER | 0 | 0

Fill the new table

```
INSERT INTO reviews (review_id, book_id, reviewer, stars) VALUES (1, 1, 'Aisha', 5),        (2, 7, 'Ben', 4),        (3, 1, 'Carlos', 4);
SELECT * FROM reviews;
```

Expected result:
review_id | book_id | reviewer | stars
1 | 1 | Aisha | 5
2 | 7 | Ben | 4
3 | 1 | Carlos | 4

ALTER TABLE

```
ALTER TABLE reviews ADD COLUMN comment TEXT;
SELECT * FROM reviews;
```

Expected result:
review_id | book_id | reviewer | stars | comment
1 | 1 | Aisha | 5 | NULL
2 | 7 | Ben | 4 | NULL
3 | 1 | Carlos | 4 | NULL

DROP TABLE

```
CREATE TABLE old_notes (note TEXT); DROP TABLE old_notes;
SELECT name FROM sqlite_master WHERE type = 'table';
```

Error message (expected):
error
You can only execute one statement at a time.

Expected result:
name
books
members
loans
reviews

Use your new table

```
SELECT b.title, AVG(r.stars) AS avg_stars,        COUNT(*) AS reviews FROM reviews AS r JOIN books AS b ON r.book_id = b.id GROUP BY b.title;
```

```
Expected result:
title | avg_stars | reviews
Dog Man | 4.0 | 1
Harry Potter | 4.5 | 2
```

UPDATE two columns

```
UPDATE reviews SET stars = 5,     comment = 'Hilarious!' WHERE review_id = 2;
SELECT * FROM reviews;
```

```
Expected result:
review_id | book_id | reviewer | stars | comment
1 | 1 | Aisha | 5 | NULL
2 | 7 | Ben | 5 | Hilarious!
3 | 1 | Carlos | 4 | NULL
```

UPDATE many rows on purpose

```
UPDATE loans SET return_date = '2026-06-10' WHERE return_date IS NULL;
SELECT * FROM loans;
```

```
Expected result:
loan_id | book_id | member_id | loan_date | return_date
1 | 1 | 1 | 2026-01-05 | 2026-01-19
2 | 3 | 1 | 2026-02-10 | 2026-06-01
3 | 2 | 2 | 2026-01-20 | 2026-02-03
4 | 4 | 3 | 2026-03-01 | 2026-03-15
5 | 1 | 3 | 2026-04-02 | 2026-06-10
6 | 7 | 4 | 2026-04-10 | 2026-04-24
7 | 7 | 2 | 2026-05-01 | 2026-06-10
```

Check before you delete

```
SELECT * FROM reviews WHERE stars < 5;
```

```
Expected result:
review_id | book_id | reviewer | stars | comment
3 | 1 | Carlos | 4 | NULL
```

DELETE with a condition

```
DELETE FROM reviews WHERE stars < 5;
SELECT * FROM reviews;
```

```
Expected result:
review_id | book_id | reviewer | stars | comment
1 | 1 | Aisha | 5 | NULL
2 | 7 | Ben | 5 | Hilarious!
```

Step 4: Challenges (try first!)

C1. Add yourself as member 7, then add a review for your favorite book.

C2. Carlos returned Harry Potter (loan 5) on 2026-06-05. Record it.

C3. Add a comment to review 1.

C4. Delete Ben's review safely (check first!).

Answers

C1. Add yourself as member 7, then add a review for your favorite book.

```
INSERT INTO members (member_id, name, city) VALUES (7, 'Me', 'My City');
INSERT INTO reviews (review_id, book_id, reviewer, stars) VALUES (10, 3, 'Me', 5);
```

C2. Carlos returned Harry Potter (loan 5) on 2026-06-05. Record it.

```
UPDATE loans SET return_date = '2026-06-05' WHERE loan_id = 5;
```

C3. Add a comment to review 1.

```
UPDATE reviews SET comment = 'Magical!' WHERE review_id = 1;
```

C4. Delete Ben's review safely (check first!).

```
SELECT * FROM reviews WHERE review_id = 2;
DELETE FROM reviews WHERE review_id = 2;
```

```
Expected result:
review_id | book_id | reviewer | stars | comment
2 | 7 | Ben | 5 | Hilarious!
```

Keep going!

Watch the whole free SQL course on Beyond School, starting with SQL Part 1: https://youtu.be/72JvYu2OtiA
Beyond School — Ready to learn in a fun way!